

10.5 Sets, Multisets, and Multimaps

We conclude this chapter by examining several additional abstractions that are closely related to the map ADT, and that can be implemented using data structures similar to those for a map.

- A **set** is an unordered collection of elements, without duplicates, that typically supports efficient membership tests. In essence, elements of a set are like keys of a map, but without any auxiliary values.
- A **multiset** (also known as a **bag**) is a set-like container that allows duplicates.
- A **multimap** is similar to a traditional map, in that it associates values with keys; however, in a multimap the same key can be mapped to multiple values. For example, the index of this book (page 714) maps a given term to one or more locations at which the term occurs elsewhere in the book.

10.5.1 The Set ADT

The Java Collections Framework defines the `java.util.Set` interface, which includes the following fundamental methods:

- `add(e)`: Adds the element *e* to *S* (if not already present).
- `remove(e)`: Removes the element *e* from *S* (if it is present).
- `contains(e)`: Returns whether *e* is an element of *S*.
- `iterator()`: Returns an iterator of the elements of *S*.

There is also support for the traditional mathematical set operations of **union**, **intersection**, and **subtraction** of two sets *S* and *T*:

$$\begin{aligned} S \cup T &= \{e: e \text{ is in } S \text{ or } e \text{ is in } T\}, \\ S \cap T &= \{e: e \text{ is in } S \text{ and } e \text{ is in } T\}, \\ S - T &= \{e: e \text{ is in } S \text{ and } e \text{ is not in } T\}. \end{aligned}$$

In the `java.util.Set` interface, these operations are provided through the following methods, if executed on a set *S*:

- `addAll(T)`: Updates *S* to also include all elements of set *T*, effectively replacing *S* by $S \cup T$.
- `retainAll(T)`: Updates *S* so that it only keeps those elements that are also elements of set *T*, effectively replacing *S* by $S \cap T$.
- `removeAll(T)`: Updates *S* by removing any of its elements that also occur in set *T*, effectively replacing *S* by $S - T$.

The *template method pattern* can be applied to implement each of the methods `addAll`, `retainAll`, and `removeAll` using only calls to the more fundamental methods `add`, `remove`, `contains`, and `iterator`. In fact, the `java.util.AbstractSet` class provides such implementations. To demonstrate the technique, we could implement the `addAll` method in the context of a set class as follows:

```
public void addAll(Set<E> other) {
    for (E element : other)           // rely on iterator( ) method of other
        add(element);                 // duplicates will be ignored by add
}
```

The `removeAll` and `retainAll` methods can be implemented with similar techniques, although a bit more care is needed for `retainAll`, to avoid removing elements while iterating over the same set (see Exercise C-10.59). The efficiency of these methods for a concrete set implementation will depend on the underlying efficiency of the fundamental methods upon which they rely.

Sorted Sets

For the standard set abstraction, there is no explicit notion of keys being ordered; all that is assumed is that the `equals` method can detect equivalent elements.

If, however, elements come from a `Comparable` class (or a suitable `Comparator` object is provided), we can extend the notion of a set to define the *sorted set ADT*, including the following additional methods:

`first()`: Returns the smallest element in S .

`last()`: Returns the largest element in S .

`ceiling( $e$ )`: Returns the smallest element greater than or equal to e .

`floor( $e$ )`: Returns the largest element less than or equal to e .

`lower( $e$ )`: Returns the largest element strictly less than e .

`higher( $e$ )`: Returns the smallest element strictly greater than e .

`subSet( $e_1$ ,  $e_2$ )`: Returns an iteration of all elements greater than or equal to e_1 , but strictly less than e_2 .

`pollFirst()`: Returns and removes the smallest element in S .

`pollLast()`: Returns and removes the largest element in S .

In the Java Collection Framework, the above methods are included in a combination of the `java.util.SortedSet` and `java.util.NavigableSet` interfaces.

Implementing Sets

Although a set is a completely different abstraction than a map, the techniques used to implement the two can be quite similar. In effect, a set is simply a map in which (unique) keys do not have associated values.

Therefore, any data structure used to implement a map can be modified to implement the set ADT with similar performance guarantees. As a trivial adaption of a map, each set element can be stored as a key, and the null reference can be stored as an (irrelevant) value. Of course, such an implementation is unnecessarily wasteful; a more efficient set implementation should abandon the Entry composite and store set elements directly in a data structure.

The Java Collections Framework includes the following set implementations, mirroring similar data structures used for maps:

- `java.util.HashSet` provides an implementation of the (unordered) set ADT with a hash table.
- `java.util.concurrent.ConcurrentSkipListSet` provides an implementation of the sorted set ADT using a skip list.
- `java.util.TreeSet` provides an implementation of the sorted set ADT using a balanced search tree. (Search trees are the focus of Chapter 11.)

10.5.2 The Multiset ADT

Before discussing models for a multiset abstraction, we must carefully consider the notion of “duplicate” elements. Throughout the Java Collections Framework, objects are considered equivalent to each other based on the standard equals method (see Section 3.5). For example, keys of a map must be unique, but the notion of uniqueness allows distinct yet equivalent objects to be matched. This is important for many typical uses of maps. For example, when strings are used as keys, the instance of the string “October” that is used when inserting an entry may not be the same instance of “October” that is used when later retrieving the associated value. The call `birthstones.get("October")` will succeed in such a scenario because strings are considered equal to each other.

In the context of multisets, if we represent a collection that appears through the notion of equivalence as $\{a, a, a, a, b, c, c\}$, we must decide if we want a data structure to explicitly maintain each instance of a (because each might be distinct though equivalent), or just that there exist four occurrences. In either case, a multiset can be implemented by directly adapting a map. We can use one element from a group of equivalent occurrences as the key in a map, with the associated value either a secondary container containing all of the equivalent instances, or a count of the number of occurrences. Note that our word-frequency application in Section 10.1.2 uses just such a map, associating strings with counts.

The Java Collections Framework does not include any form of a multiset. However, implementations exist in several widely used, open source Java collections libraries. The Apache Commons defines `Bag` and `SortedBag` interfaces that correspond respectively to unsorted and sorted multisets. The Google Core Libraries for Java (named *Guava*) includes `Multiset` and `SortedMultiset` interfaces for these abstractions. Both of those libraries take the approach of modeling a multiset as a collection of elements having multiplicities, and both offer several concrete implementations using standard data structures. In formalizing the abstract data type, the `Multiset` interface of the Guava library includes the following behaviors (and more):

- `add(e)`: Adds a single occurrences of *e* to the multiset.
- `contains(e)`: Returns true if the multiset contains an element equal to *e*.
- `count(e)`: Returns the number of occurrences of *e* in the multiset.
- `remove(e)`: Removes a single occurrence of *e* from the multiset.
- `remove(e, n)`: Removes *n* occurrences of *e* from the multiset.
- `size()`: Returns the number of elements of the multiset (including duplicates).
- `iterator()`: Returns an iteration of all elements of the multiset (repeating those with multiplicity greater than one).

The multiset ADT also includes the notion of an immutable `Entry` that represents an element and its count, and the `SortedMultiset` interface includes additional methods such as `firstEntry` and `lastEntry`.

10.5.3 The Multimap ADT

Like a map, a multimap stores entries that are key-value pairs (k, v) , where *k* is the key and *v* is the value. Whereas a map insists that entries have unique keys, a multimap allows multiple entries to have the same key, much like an English dictionary, which allows multiple definitions for the same word. That is, we will allow a multimap to contain entries (k, v) and (k, v') having the same key.

There are two standard approaches for representing a multimap as a variation of a traditional map. One is to redesign the underlying data structure to allow separate entries to be stored for pairs such as (k, v) and (k, v') . The other is to map key *k* to a secondary container of all values associated with that key (e.g., $\{v, v'\}$).

Much as it is missing a formal abstraction for a multiset, the Java Collections Framework does not include any multiset interface nor implementations. However, as we will soon demonstrate, it is easy to represent a multiset by adapting other collection classes that are included in the `java.util` package.

To formalize the multimap abstract data type, we consider a simplified version of the Multimap interface included in Google's Guava library. Among its methods are the following:

- `get(k)`: Returns a collection of all values associated with key *k* in the multimap.
- `put(k, v)`: Adds a new entry to the multimap associating key *k* with value *v*, without overwriting any existing mappings for key *k*.
- `remove(k, v)`: Removes an entry mapping key *k* to value *v* from the multimap (if one exists).
- `removeAll(k)`: Removes all entries having key equal to *k* from the multimap.
- `size()`: Returns the number of entries of the multiset (including multiple associations).
- `entries()`: Returns a collection of all entries in the multimap.
- `keys()`: Returns a collection of keys for all entries in the multimap (including duplicates for keys with multiple bindings).
- `keySet()`: Returns a nonduplicative collection of keys in the multimap.
- `values()`: Returns a collection of values for all entries in the multimap.

In Code Fragments 10.16 and 10.17, we provide an implementation of a class, HashMultimap, that uses a `java.util.HashMap` to map each key to a secondary `ArrayList` of all values that are associated with the key. For brevity, we omit the formality of defining a Multimap interface, and we provide the `entries()` method as the only form of iteration.

```

1 public class HashMultimap<K,V> {
2     Map<K,List<V>> map = new HashMap<>(); // the primary map
3     int total = 0; // total number of entries in the multimap
4     /** Constructs an empty multimap. */
5     public HashMultimap() { }
6     /** Returns the total number of entries in the multimap. */
7     public int size() { return total; }
8     /** Returns whether the multimap is empty. */
9     public boolean isEmpty() { return (total == 0); }
10    /** Returns a (possibly empty) iteration of all values associated with the key. */
11    Iterable<V> get(K key) {
12        List<V> secondary = map.get(key);
13        if (secondary != null)
14            return secondary;
15        return new ArrayList<>(); // return an empty list of values
16    }

```

Code Fragment 10.16: An implementation of a multimap as an adaptation of classes from the `java.util` package. (Continues in Code Fragment 10.17.)

```

17  /** Adds a new entry associating key with value. */
18  void put(K key, V value) {
19      List<V> secondary = map.get(key);
20      if (secondary == null) {
21          secondary = new ArrayList<>();
22          map.put(key, secondary);    // begin using new list as secondary structure
23      }
24      secondary.add(value);
25      total++;
26  }
27  /** Removes the (key,value) entry, if it exists. */
28  boolean remove(K key, V value) {
29      boolean wasRemoved = false;
30      List<V> secondary = map.get(key);
31      if (secondary != null) {
32          wasRemoved = secondary.remove(value);
33          if (wasRemoved) {
34              total--;
35              if (secondary.isEmpty())
36                  map.remove(key);    // remove secondary structure from primary map
37          }
38      }
39      return wasRemoved;
40  }
41  /** Removes all entries with the given key. */
42  Iterable<V> removeAll(K key) {
43      List<V> secondary = map.get(key);
44      if (secondary != null) {
45          total -= secondary.size();
46          map.remove(key);
47      } else
48          secondary = new ArrayList<>();    // return empty list of removed values
49      return secondary;
50  }
51  /** Returns an iteration of all entries in the multimap. */
52  Iterable<Map.Entry<K,V>> entries() {
53      List<Map.Entry<K,V>> result = new ArrayList<>();
54      for (Map.Entry<K,List<V>> secondary : map.entrySet()) {
55          K key = secondary.getKey();
56          for (V value : secondary.getValue())
57              result.add(new AbstractMap.SimpleEntry<K,V>(key,value));
58      }
59      return result;
60  }
61  }

```

Code Fragment 10.17: An implementation of a multimap as an adaptation of classes from the `java.util` package. (Continued from Code Fragment 10.16.)